

CS 305: THEORY OF COMPUTATION

Module 1: Automata, Formal Languages & Modern Systems

Weeks 1–5: From Mathematical Foundations to Systems Security

Department of Computer Science

School of Technology

Ghana Institute of Management and Public Administration (GIMPA)

Level 300 Undergraduate · Semester 1

Module 1 Roadmap: Theory ↔ Cutting-Edge Systems

Theoretical Foundations (Sipser Ch. 1)

- **Week 1:** Formal Languages & Chomsky Hierarchy
- **Week 2:** Deterministic Finite Automata (DFAs)
- **Week 3:** NFAs & Powerset Construction Algorithm
- **Week 4:** Regular Expressions & GNFA State Elimination
- **Week 5:** Non-regular Languages & The Pumping Lemma

Research Literature & System Spotlights

- **Week 1: ICLR 2023:** Neural Networks & Chomsky Hierarchy
- **Week 2: ACM SIGCOMM:** 100 Gbps Deep Packet Inspection
- **Week 3: CACM:** Linear vs. Exponential Regex Engines
- **Week 4: USENIX Security 2018:** Catastrophic ReDoS
- **Week 5:** Bounded Memory in Edge Hardware & Transformers

Central Thesis: Finite automata are not historical curiosities; they govern line-rate packet routers, compilers, and LLM reasoning boundaries.

Week 1: Core Mathematical Foundations

Alphabets, Strings, and Languages (Sipser Ch. 0)

- **Alphabet (Σ):** A non-empty, finite set of symbols (e.g., binary $\Sigma = \{0, 1\}$).
 - ▶ **Modern AI Connection: LLM Tokenization Alphabets** (e.g., GPT/Llama BPE vocabulary $|\Sigma| \approx 100\text{k}$ token IDs).
- **String (w):** A finite sequence of symbols chosen from Σ . Length $|w|$ denotes symbol count. The empty string is denoted ε ($|\varepsilon| = 0$).
- **Language (L):** Any set of strings formed over an alphabet ($L \subseteq \Sigma^*$).
- **Kleene Star (Σ^*):** The set of all possible finite strings over Σ , including ε : $\Sigma^* = \bigcup_{k=0}^{\infty} \Sigma^k$.

Concrete Language Instance Let $\Sigma = \{0, 1\}$. The language of all binary strings with an even number of zeros is defined as:

$$L = \{w \in \Sigma^* \mid |w|_0 \equiv 0 \pmod{2}\} = \{\varepsilon, 1, 11, 00, 010, 1001, \dots\}.$$

Week 1 Spotlight: Chomsky Hierarchy in Modern AI

Seminal Literature (ICLR 2023 Spotlight): Delétang et al. (Google DeepMind), *“Neural Networks and the Chomsky Hierarchy,”* ICLR 2023.

The Chomsky Hierarchy (1956)

Type 0: Recursively Enumerable **Turing / CoT LLMs**

Unrestricted Grammars ($\alpha \rightarrow \beta$) · Arbitrary computation

Type 1: Context-Sensitive **Linear Bounded Automata**

Context-Sensitive Grammars ($\alpha A \beta \rightarrow \alpha \gamma \beta$) · Bounded tape

Type 2: Context-Free **Pushdown Automata (PDAs)**

Context-Free Grammars ($A \rightarrow \gamma$) · Stack memory, AST parsers

Type 3: Regular Languages **Finite Automata (DFAs/NFAs)**

Regular Grammars ($A \rightarrow aB$) · Finite memory, tokenizers, regex

Strict Set Inclusion: $\mathcal{L}(\text{Type 3}) \subsetneq \mathcal{L}(\text{Type 2}) \subsetneq \mathcal{L}(\text{Type 1}) \subsetneq \mathcal{L}(\text{Type 0})$

Why Finite Models Fail on Recursive Syntax

- **Limits of Finite Attention:** Standard Transformers without external scratchpads fail to generalize on non-regular counting tasks ($a^n b^n c^n$).
- **Theoretical Mirror:** Neural architectures strictly obey the classical 1956 Chomsky hierarchy boundaries in out-of-distribution evaluation.
- **Chain-of-Thought (CoT):** Generating reasoning tokens elevates bounded models to Type 0 Turing computation!

Week 2: Deterministic Finite Automata (DFA)

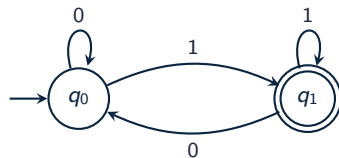
Formal 5-Tuple Definition (Sipser Def 1.5)

A **DFA** is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$:

1. Q : Finite set of internal **states**.
2. Σ : Finite input **alphabet**.
3. $\delta : Q \times \Sigma \rightarrow Q$: **Transition function**.
4. $q_0 \in Q$: Designated **start state**.
5. $F \subseteq Q$: Set of **accept states**.

Determinism Invariant: For every pair $(q, a) \in Q \times \Sigma$, exactly one transition exists.

DFA State Diagram: Strings Ending in '1'



Trace on $w = 0101$:

$q_0 \xrightarrow{0} q_0 \xrightarrow{1} q_1 \xrightarrow{0} q_0 \xrightarrow{1} q_1 \in F$ (**Accepted**).

Complexity: Strictly $\mathcal{O}(|w|)$ linear time, $\mathcal{O}(1)$ space.

Week 2 Spotlight: High-Speed Deep Packet Inspection

Seminal Literature (ACM SIGCOMM):

Kumar et al. (Univ. of Wisconsin / Cisco), *"Algorithms to Accelerate Multiple Regular Expression Matching for Deep Packet Inspection,"* ACM SIGCOMM CCR.

The Line-Rate Hardware Challenge

- **Extreme Bandwidth:** Modern edge routers and firewalls must scan **100+ Gbps** in real time without dropping frames.
- **Massive Rule Sets:** Systems like Snort compare packet payloads against $\geq 10,000$ threat patterns simultaneously.
- **State Explosion:** Combining N regex rules creates a composite DFA with exponential states ($|Q| \sim 2^N$).

The Automata Solution: Deferred D^2FA

- **$\mathcal{O}(1)$ Deterministic Lookup:** The hardware engine processes each incoming byte in strictly **one memory cycle**.
- **Deferred DFA (D^2FA):** Eliminates redundant transitions via default failure pointers, reducing memory by $> 95\%$.
- **System Guarantee:** Zero packet buffering, deterministic line-rate security filtering.

Week 3: Nondeterministic Finite Automata (NFA)

Formal Definition (Sipser Def 1.16)

An **NFA** is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$ where:

$$\delta : Q \times \Sigma_{\varepsilon} \rightarrow \mathcal{P}(Q), \quad \Sigma_{\varepsilon} = \Sigma \cup \{\varepsilon\}$$

- **Branching:** Transitions to subsets of states.
- **ε -Transitions:** State jumps without input.
- **Acceptance:** Accepted if *at least one* path terminates in an accept state $q \in F$.

Fundamental Equivalence Theorem

Theorem (Sipser Thm 1.39):

Every NFA has an equivalent DFA recognizing the exact same formal language.

$$\mathcal{L}(\text{NFA}) = \mathcal{L}(\text{DFA}) = \text{Regular Languages}$$

- **Proof Method:** Powerset (Subset) Construction.
- **Expressive Equivalence:** Zero added power, but exponential descriptive conciseness ($|Q_{\text{NFA}}| \ll |Q_{\text{DFA}}|$).

Week 3: The Powerset (Subset) Construction Algorithm

Algorithm: Convert NFA $N = (Q, \Sigma, \delta, q_0, F)$ to Equivalent DFA $M = (Q', \Sigma, \delta', q'_0, F')$

1. **State Space Construction:** The DFA state set is the power set of NFA states:

$$Q' = \mathcal{P}(Q) \implies |Q'| \leq 2^{|Q|}.$$

2. **DFA Start State:** Include the initial state and all states reachable via ϵ -transitions:

$$q'_0 = E(\{q_0\}), \quad \text{where } E(R) \text{ is the } \epsilon\text{-closure of state set } R.$$

3. **DFA Transition Function:** For each subset $R \in Q'$ and symbol $a \in \Sigma$:

$$\delta'(R, a) = \bigcup_{r \in R} E(\delta(r, a)).$$

4. **DFA Accept States:** Any subset containing at least one original accept state:

$$F' = \{R \in Q' \mid R \cap F \neq \emptyset\}.$$

Takeaway: Nondeterminism provides exponential descriptive conciseness without increasing computational power.

Week 4: Regular Expressions & Kleene's Theorem

Inductive Definition of Regular Expressions (Sipser Def 1.52) A string R is a regular expression over Σ if R is:

1. **Base Cases:** $a \in \Sigma$ ($\{a\}$), ε ($\{\varepsilon\}$), or $\emptyset$ ($\emptyset$).
2. **Inductive Operations:** Given regular expressions R_1, R_2 :
 - ▶ **Union:** $R_1 \cup R_2$ (or $R_1 \mid R_2$), recognizing $L(R_1) \cup L(R_2)$.
 - ▶ **Concatenation:** $R_1 \circ R_2$ (or $R_1 R_2$), recognizing $\{uv \mid u \in L(R_1), v \in L(R_2)\}$.
 - ▶ **Star (Kleene Closure):** R_1^* , recognizing $\bigcup_{k=0}^{\infty} L(R_1)^k$.

Kleene's Theorem (1956)

Language L is Regular $\iff L$ is described by a Regex $\iff L$ is recognized by a DFA / NFA.

Equivalence Pipeline: Regex $\xrightarrow{\text{Thompson's Alg.}}$ NFA $\xrightarrow{\text{Subset Const.}}$ DFA $\xrightarrow{\text{State Elimination}}$ Regex.

Week 4 Spotlight: Catastrophic ReDoS Vulnerabilities

Seminal Literature (USENIX Security 2018):

Staicu & Pradel (TU Darmstadt), "Freezing the Web: A Study of ReDoS Vulnerabilities in JavaScript-based Web Servers," USENIX Security Symposium 2018.

Vulnerability: Backtracking Engines

- **Engine Flaw:** Python re, Node.js (V8), and Java implement regex via recursive depth-first backtracking.
- **Pathological Regex:** A nested quantifier like $(a^+)^b$ on input string a^n requires $\mathcal{O}(2^n)$ evaluations.
- **Denial of Service (ReDoS):** A 35-character malicious HTTP payload freezes Node.js event loops for 10+ hours (100% CPU spike).

Solution: Automata-Theoretic Execution

- **The Thompson NFA Remedy:** Modern secure engines (Google RE2, Rust regex) abandon backtracking.
- **Simultaneous Set Tracking:** The engine tracks all active NFA states in lockstep:

$\mathcal{O}(|w| \cdot |Q|)$ strictly linear worst-case time!

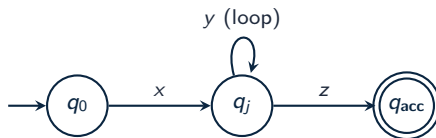
- **Immunity:** Completely immune to algorithmic complexity attacks by theoretical construction.

Week 5: Limits of Finite Automata & Pigeonhole Principle

The Pigeonhole Principle & Loops

- Let M be a DFA with p states ($|Q| = p$).
- If M processes any string $s \in L(M)$ with length $|s| \geq p$, the path visits at least $p + 1$ states.
- By the **Pigeonhole Principle**, at least one state $q_j \in Q$ must be visited **twice**.
- The computation necessarily contains a **directed cycle (loop)**.

Automata Cycle Decomposition



Decompose string $s = xyz$ where:

- x : prefix driving $q_0 \rightarrow q_j$.
- y : loop cycle driving $q_j \rightarrow q_j$ ($|y| > 0$).
- z : suffix driving $q_j \rightarrow q_{acc}$.

Core Consequence: The cycle y can be pumped $i = 0, 1, 2, \dots$ times; the automaton cannot prevent xy^iz from being accepted, proving language regularity.

Week 5: The Pumping Lemma for Regular Languages

Formal Theorem Statement (Sipser Thm 1.70) If L is a regular language, then there exists a pumping length $p \geq 1$ such that every string $s \in L$ with $|s| \geq p$ can be divided into $s = xyz$ satisfying three conditions:

1. $xy^iz \in L$ for each $i \geq 0$ (Pumping Property),
2. $|y| > 0$ (Non-empty Loop),
3. $|xy| \leq p$ (Loop Occurs within First p Characters).

Adversarial Proof Template: Showing $L = \{0^n1^n \mid n \geq 0\}$ is Non-Regular

1. **Proof by Contradiction:** Assume L is regular. The Pumping Lemma guarantees a pumping length p .
2. **Select String:** Choose $s = 0^p1^p \in L$. Observe that $|s| = 2p \geq p$.
3. **Constrain Substring y :** Since $|xy| \leq p$, substring y lies entirely within the initial block of zeros: $y = 0^k$ for $k \geq 1$.
4. **Pump Down ($i = 0$):** Consider $s' = xy^0z = xz$. Symbol count: $|s'|_0 = p - k \neq p = |s'|_1$.
5. **Contradiction:** $s' \notin L$, violating Condition 1 $\implies L$ is strictly non-regular!

Module 1 Summary & Looking Ahead to Module 2

Module 1 Key Scientific Takeaways

- **Model Equivalence:** DFAs, NFAs, and Regular Expressions possess identical expressive capability ($\mathcal{L}_{\text{Reg}}$).
- **The Boundary of Memory:** Finite automata cannot track unbounded state; they are strictly incapable of counting (0^n1^n) or verifying balanced syntax.
- **Systems Impact:** Automata algorithms deliver deterministic $\mathcal{O}(n)$ performance in network routing and protect web services from ReDoS.

Looking Ahead: Module 2 (Weeks 6–8)

- **Parsing Programming Languages:** Compilers, JSON parsers, and LLM structured decoding require nested hierarchical syntax.
- **The Stack Upgrade:** Augmenting finite automata with unbounded LIFO memory:
 $\text{Finite Control} + \text{Stack} \implies \text{Pushdown Automata (PDAs)}$
- **Grammar Formalism:** Context-Free Grammars (CFGs) and Chomsky Normal Form.

Questions & Discussion

Preparation for Lab 1: Simulating DFAs & NFAs with JFLAP / Python Automata